

Learn Git Branching

A Complete, Level-by-Level Study Guide

Concepts | Commands | Examples | Real-world use | Common mistakes

Companion to learngitbranching.js.org - Main levels 1-5 and Remote levels 1-2

How to use this guide

This guide follows the exact order of the levels on learngitbranching.js.org. Each topic is split into the same sections so you can revise quickly:

- **What it is** - the concept in plain language.
- **Commands and example** - what to type, with a small commit-graph drawing (Cn = commit, '*' = where HEAD is, lines point to parents).
- **Where it is mostly used** - real-world situations.
- **Common mistakes** - the traps beginners and even experienced developers fall into.
- **Level hint** - a nudge for the matching site level, without spoiling the full solution.

Site vs real Git. The site simplifies a few things: commit IDs are C0, C1... instead of hashes; there is no staging area (no `git add`); remote branches appear as `o/main` instead of `origin/main`; and `git fakeTeamwork` exists only on the site. These differences are pointed out where they matter.

Useful site commands: `levels` (level picker), `reset` (restart level), `undo`, `hint`, `show goal`, `show solution`, `sandbox` (free play), `objective`.

Contents

Main 1 - Introduction Sequence	1.1 Git Commits 1.2 Branching in Git 1.3 Merging in Git 1.4 Rebase Introduction
Main 2 - Ramping Up	2.1 Detach yo' HEAD 2.2 Relative Refs (^) 2.3 Relative Refs #2 (~) and branch forcing 2.4 Reversing Changes in Git (reset vs revert)
Main 3 - Moving Work Around	3.1 Cherry-pick Intro 3.2 Interactive Rebase Intro
Main 4 - A Mixed Bag	4.1 Grabbing Just 1 Commit 4.2 Juggling Commits 4.3 Juggling Commits #2 4.4 Git Tags 4.5 Git Describe
Main 5 - Advanced Topics	5.1 Rebasing over 9000 times 5.2 Multiple parents 5.3 Branch Spaghetti
Remote 1 - Push & Pull: Git Remotes	R1.1 Clone Intro R1.2 Remote Branches R1.3 Git Fetchin' R1.4 Git Pullin' R1.5 Faking Teamwork R1.6 Git Pushin' R1.7 Diverged History R1.8 Locked Main
Remote 2 - To Origin and Beyond: Advanced Remotes	R2.1 Push Main! R2.2 Merging with remotes R2.3 Remote Tracking R2.4 Git push arguments R2.5 Git push arguments - Expanded R2.6 Fetch arguments R2.7 Source of nothing R2.8 Pull arguments
Appendices	A. Merge vs Rebase B. Recovery toolkit C. Command cheat sheet D. Self-test questions

Key vocabulary

Repository	The project plus its full history (the hidden .git folder).
Commit	A snapshot of the project with a parent pointer and a message.
Branch	A movable pointer to a commit.
HEAD	Where you currently are - usually a branch, sometimes a commit (detached).
Working directory	The files you edit on disk.
Staging area (index)	The set of changes prepared for the next commit (git add).
Remote / origin	Another copy of the repo, usually on GitHub/GitLab; 'origin' is the default name.
Remote-tracking branch	origin/main (o/main on site): your record of where the remote branch was.
Upstream	The remote branch a local branch tracks for push/pull.
Fast-forward	Moving a branch pointer forward because no divergent work exists.
Rewriting history	Replacing commits with new ones (rebase, amend, reset) - avoid on shared branches.

The single most important idea: Git history is a graph of commits. Branches, tags and HEAD are just labels pointing into that graph. Almost every command either *creates commits* or *moves labels*. When you are confused, ask: 'Which commits exist, and where are the labels?'

Main 1 - Introduction Sequence

The foundation: what a commit is, what a branch really is, and the two ways to combine work (merge and rebase). Everything later on the site builds on these four ideas.

1.1 Git Commits

Site level: *Introduction to Git Commits*

What it is

A **commit** is a saved snapshot of your whole project at a moment in time. Git stores it efficiently (only what changed is new data), and every commit remembers its **parent** - the commit it was built on. Following parents backwards gives you the project's history. In real Git each commit has a long SHA-1 hash ID such as a1b2c3d; the site simplifies these to C0, C1, C2 and so on.

Commands and example

```
git add file.js          # stage changes (real Git; not needed on the site)
git commit -m "Add login form"
git log --oneline --graph # see history
```

```
Before: C0 <- C1 (main*)
After : C0 <- C1 <- C2 (main*)
```

The arrow points to the parent. '*' marks where HEAD is.

Where it is mostly used

Every day, for every meaningful unit of work: a bug fix, a new function, a config change. Small, focused commits make code review, debugging (git bisect) and undoing mistakes far easier.

Common mistakes

- Forgetting to **stage** files with `git add` in real Git, so the commit is empty or incomplete (the site hides this step, so it is easy to forget when you move to a real terminal).
- Making one giant commit with unrelated changes. It becomes impossible to revert just one part.
- Vague messages like "fix" or "update". Write what and why: "Fix crash when cart is empty".
- Committing secrets (.env files, API keys) or generated files (node_modules, build output). Use a .gitignore.
- Committing while on the wrong branch - always check which branch HEAD is on first (`git status`).

Level hint: The level only needs two commits. Watch how the main* label moves forward with each one.

1.2 Branching in Git

Site level: *Branching in Git*

What it is

A **branch** is just a lightweight, movable **pointer to a commit**. It is not a copy of your files. Creating a branch is instant and costs almost nothing, which is why Git users branch early and often. When you commit, the branch that HEAD is on moves forward to the new commit.

Commands and example

```
git branch bugFix      # create pointer (you STAY on current branch)
git checkout bugFix    # switch to it
git checkout -b bugFix  # create + switch in one step
git switch -c bugFix    # modern equivalent
```

```
git branch newImage; git commit
C0 <- C1 <- C2 (main*)      newImage still at C1 <- surprise!

git checkout newImage; git commit
C0 <- C1 <- C2 (main)
      \
      C3 (newImage*)
```

Where it is mostly used

Feature development (feature/login), bug fixes (fix/cart-crash), experiments, and release preparation. Almost every team workflow (GitHub Flow, Git Flow, trunk-based) is built on short-lived branches.

Common mistakes

- The classic: running `git branch x` and then committing, expecting the commit to land on x. `git branch` only creates; it does **not** switch. The commit goes to your current branch.
- Thinking a branch is a folder or a full copy of the code. It is a 41-byte pointer.
- Letting branches live for weeks, so they drift far from main and cause painful merges.
- Poor names (test, new, temp2). Use descriptive names like feature/search-filter.
- Deleting a branch with unmerged work using `git branch -D` (capital D forces it).

Level hint: Look for the '*' on the branch label in the visual - that tells you where your next commit will go.

1.3 Merging in Git

Site level: *Merging in Git*

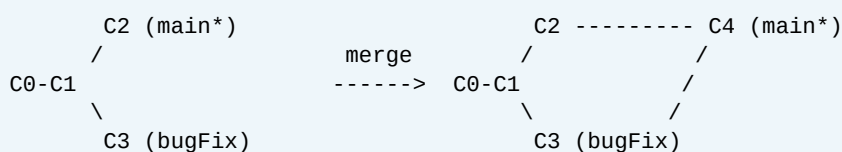
What it is

Merging combines the work of two branches. `git merge X` brings X's commits **into the branch you are currently on**. If both sides have new commits, Git creates a special **merge commit** with two parents. If your branch has no new commits of its own, Git simply slides the pointer forward - a **fast-forward** merge - and no new commit is created.

Commands and example

```
git checkout main
git merge bugFix      # main now contains bugFix's work

git checkout bugFix
git merge main        # fast-forward bugFix up to main
```



C4 is a merge commit with two parents: C2 and C3.

Where it is mostly used

Integrating a finished feature into main, pulling the latest main into your feature branch, and completing pull requests on GitHub/GitLab (the green "Merge" button). Merge preserves the true history of how work happened.

Common mistakes

- Merging in the **wrong direction**: being on bugFix and running `git merge main` when you meant to put bugFix into main. Always check where HEAD is.
- Not understanding fast-forward: being surprised that no merge commit appeared (use `--no-ff` if your team wants one).
- Panicking at merge conflicts. Open the files, fix the <<<<<<< / >>>>>>> sections, `git add`, then `git commit`. Or back out with `git merge --abort`.
- Merging an outdated local main - run `git pull` first.
- Merging with uncommitted changes in the working directory, which can block or complicate the merge.

Level hint: After merging bugFix into main, notice that bugFix is behind main. The level wants you to see why you might merge main back into bugFix.

1.4 Rebase Introduction

Site level: *Rebase Introduction*

What it is

Rebasing is the second way to combine work. It takes the commits of your branch and **replays (copies) them on top of another base**, producing a clean, straight-line history. The copies are new commits (the site shows them as C3') with new hashes; the originals become unreachable and are eventually cleaned up.

Commands and example

```
git checkout bugFix
git rebase main          # replay bugFix's commits on top of main

git checkout main
git rebase bugFix        # main is an ancestor -> just fast-forwards

git rebase main bugFix   # two-arg form: rebase bugFix onto main
                        # (checks bugFix out first)
```



Where it is mostly used

Updating a feature branch with the latest main before opening a pull request, keeping a project's history linear and readable, and 'Rebase and merge' on GitHub. Many teams prefer rebase for local, unshared work.

Common mistakes

- Breaking the **golden rule**: never rebase commits that others already have (pushed, shared branches). It rewrites history and forces teammates into a mess.
- Thinking commits are 'moved'. They are copied with new IDs - which is why a force push is needed if the branch was already pushed.
- Running it from the wrong branch: `git rebase X` moves the *current* branch onto X, not the other way round.
- Forgetting to fast-forward main afterwards, so main still does not contain the work.
- Getting lost during conflicts. Resolve, `git add`, then `git rebase --continue`; or escape with `git rebase --abort`.

Level hint: Compare the final graph with the merge level: same content, but no merge commit and a straight line.

Main 2 - Ramping Up

Now you learn to move around the commit tree precisely: what HEAD is, how to point at commits relative to others, and how to undo work safely.

2.1 Detach yo' HEAD

Site level: *Detach yo' HEAD*

What it is

HEAD is Git's name for 'where you are right now'. Normally HEAD points to a **branch** (which points to a commit). If you check out a commit directly, HEAD points straight at that commit - this is **detached HEAD** state.

Commands and example

```
cat .git/HEAD          # ref: refs/heads/main (attached)
git checkout C1         # HEAD -> C1 directly (detached)
git symbolic-ref HEAD  # errors when detached
git switch -           # go back to previous branch
```

```
Attached:  HEAD -> main -> C1
Detached:  HEAD -> C1          (main still points wherever it was)
```

Where it is mostly used

Inspecting an old version of the code, running tests on a specific commit, `git bisect` for hunting bugs, and CI/CD pipelines, which usually check out an exact commit rather than a branch.

Common mistakes

- Making commits in detached HEAD, then switching branches. Those commits belong to no branch and seem to vanish. Save them first with `git switch -c rescue`.
- Panicking at the 'detached HEAD' warning - it is normal and harmless if you are only looking around.
- Not knowing the rescue tool: `git reflog` lists where HEAD has been, so lost commits can be recovered.

Level hint: Check out the commit itself (its ID), not the branch name. The HEAD label will split away from the branch label.

2.2 Relative Refs (^)

Site level: *Relative Refs (^)*

What it is

Typing hashes is tedious, so Git lets you point to commits **relative** to a branch or HEAD. The caret `^` means 'the parent of'. Stack them to go further back: `main^^` is the grandparent. In real Git you also only need enough of a hash to be unique (e.g. `fed2` instead of the full 40 characters).

Commands and example

```
git checkout main^      # parent of main
git checkout main^^     # grandparent
git checkout HEAD^      # move up one from where you are
git show HEAD^          # inspect the previous commit
```

```
C0 <- C1 <- C2 <- C3 (main)
      ^      ^
      main^^ main^
```

Where it is mostly used

Quick navigation, comparing changes (`git diff HEAD^ HEAD`), and referring to 'the commit before this one' in scripts and resets.

Common mistakes

- Thinking `^2` means 'two commits back'. It means the **second parent** of a merge commit (see 5.2). Use `^^` or `~2` for two back.
- On Windows cmd, `^` is an escape character; quote it ("`HEAD^`") or use `~`.
- Forgetting that relative refs are resolved from the current position, so repeating `git checkout HEAD^` keeps walking back.

Level hint: The level wants you to reach a commit using `^` from a branch name, not by typing its ID.

2.3 Relative Refs #2 (~) and branch forcing

Site level: *Relative Refs #2 (~)*

What it is

The tilde `~N` means 'go back N parents' - `HEAD~4` equals `HEAD^^^^`. This level also introduces **branch forcing**: `git branch -f` moves a branch pointer to any commit you choose.

Commands and example

```
git checkout HEAD~4
git branch -f main HEAD~3      # move main 3 commits behind HEAD
git branch -f bugFix C6        # move bugFix to a specific commit
```

C0-C1-C2-C3-C4 (main, HEAD)

```
git branch -f main HEAD~3 (from another branch)
C0-C1 (main) -C2-C3-C4
```

Where it is mostly used

Repositioning branches during cleanup, pointing a branch at a known-good commit, and in scripts that need 'N commits ago'.

Common mistakes

- Using `-f` carelessly: commits that no branch points to become invisible (recoverable only via `reflog` for a while).
- Trying to force the branch you are currently on. Real Git refuses; use `git reset` for that.
- Mixing up `~` (walk first parents) and `^N` (choose among parents).

Level hint: You can mix: the target is often reachable in one command like `git branch -f name HEAD~1` or `git checkout main^`.

2.4 Reversing Changes in Git (reset vs revert)

Site level: *Reversing Changes in Git*

What it is

Two ways to undo. **git reset** moves the current branch backwards, as if the later commits never happened - it **rewrites history**, fine for local work. **git revert** creates a **new commit** that applies the opposite changes, so history is preserved and it is safe for shared branches.

Commands and example

```
git reset HEAD~1      # local branch: drop last commit
                      # real Git default (--mixed) keeps the changes
                      # as unstaged edits in your files
git reset --soft HEAD~1 # keep changes staged
git reset --hard HEAD~1 # DISCARD changes completely

git revert HEAD        # shared branch: new commit C2' undoes C2
```

```
reset:  C0-C1-C2 (local*)  ->  C0-C1 (local*)
revert:  C0-C1-C2 (pushed*) ->  C0-C1-C2-C2' (pushed*)
                                   (C2' is the inverse of C2)
```

Where it is mostly used

reset: fixing your last local commit before pushing, unstaging files. revert: undoing a bad change that already reached main or production - the standard way on teams.

Common mistakes

- Using **reset on a pushed branch** - your history now disagrees with the remote, and fixing it requires a force push that hurts teammates.
- Running `git reset --hard` and losing uncommitted work permanently (uncommitted changes are NOT in the reflog).
- Expecting revert to delete the commit. It stays in history; only its effect is cancelled.
- Reverting a merge commit without `-m 1` (Git needs to know which parent is the 'mainline').
- Note: the site's reset simply moves the branch; remember the `--soft` / `--mixed` / `--hard` differences in real Git.

Level hint: Use reset on the branch named 'local' and revert on the branch named 'pushed' - the names are the hint.

Main 3 - Moving Work Around

The tools for saying exactly which commits should go where: cherry-pick and interactive rebase.

3.1 Cherry-pick Intro

Site level: *Cherry-pick Intro*

What it is

`git cherry-pick` copies specific commits and places them on top of your current position (HEAD). Use it when you know **exactly which commits** you want, by name.

Commands and example

```
git checkout main
git cherry-pick C2 C4      # copies appear as C2' and C4' on main
git cherry-pick A..B      # real Git: range (excludes A)
```

```

C0-C1 (main*)                C0-C1-C2'-C4' (main*)
  \                          \
   \      cherry-pick         \
    C2-C3-C4 (side)  -----> C2-C3-C4 (side)
```

Where it is mostly used

Back-porting a hotfix from main to a release branch (release/2.1), rescuing one good commit from an abandoned branch, or moving a commit that was made on the wrong branch.

Common mistakes

- Cherry-picking a commit and later merging the whole branch too - you may get duplicate changes or confusing conflicts.
- Picking commits in the wrong order when later ones depend on earlier ones.
- Cherry-picking a merge commit without `-m`.
- Being on the wrong branch - the copies go wherever HEAD is.
- Overusing it as a replacement for proper merging; it scatters duplicate commits across branches.

Level hint: List the commits in the order you want them to appear.

3.2 Interactive Rebase Intro

Site level: *Interactive Rebase Intro*

What it is

`git rebase -i` opens an editor listing recent commits so you can **reorder, drop, squash, edit or reword** them before replaying. Use it when you **don't know the IDs** or want to tidy up. On the site a drag-and-drop window appears where you can reorder and omit commits.

Commands and example

```
git rebase -i HEAD~4

# editor (oldest at the top!):
pick    a1 Add model
squash  b2 Fix typo in model    # combine into previous
drop    c3 Debug prints        # remove
reword  d4 Add view            # change message
# also: edit, fixup
```

```

C0-C1-C2-C3-C4 (main*)
  rebase -i HEAD~4, reorder to C3,C1, drop C2,C4
C0-C3'-C1' (main*)
```

Where it is mostly used

Cleaning up messy local commits before a pull request: squashing 'wip' commits, removing debug commits, fixing messages, reordering logically. Very common in open-source contributions.

Common mistakes

- Choosing too big a range and rewriting commits that are already pushed/shared.
- Reading the list backwards - in the editor the **oldest commit is at the top** (opposite of git log).
- Squashing into the wrong commit (squash merges into the one *above* it).
- Deleting a line accidentally drops that commit.
- Forgetting the escape hatch: `git rebase --abort`.

Level hint: Use `git rebase -i` with a relative ref like `HEAD~4`, then `reorder/omit` to match the goal.

Main 4 - A Mixed Bag

Realistic scenarios that combine earlier tools, plus tags and git describe.

4.1 Grabbing Just 1 Commit

Site level: *Locally stacked commits*

What it is

Scenario: while hunting a bug you added debug and print-statement commits, then the actual fix. You want **only the fix** on main. Solve with either `git cherry-pick` (grab the fix) or `git rebase -i` (drop the debug commits).

Commands and example

```
git checkout main
git cherry-pick C4          # only the fix
# or
git rebase -i main bugFix # drop debug commits, then fast-forward main
```

```
C0-C1 (main)                C0-C1-C4' (main*)
      \
      C2(debug)-C3(print)-C4(fix) (bugFix)
```

Where it is mostly used

Any time experimental or temporary commits sit alongside the real fix - extremely common during debugging.

Common mistakes

- Merging the whole bugFix branch and shipping debug logging to production.
- Forgetting that main must end up pointing at the fix, not just have it somewhere.
- Avoiding the problem entirely is better: use `git stash` or don't commit debug code at all.

Level hint: Either tool works. Main just needs the fix commit on top.

4.2 Juggling Commits

Site level: *Juggling Commits*

What it is

Scenario: you need to change an **older** commit, but `git commit --amend` only edits the **latest** commit. Strategy: reorder so the target is on top, amend it, reorder back, then move main.

Commands and example

```
git rebase -i HEAD~2      # move target commit to top
git commit --amend        # change it (C2 -> C2'')
git rebase -i HEAD~2      # restore original order
git branch -f main HEAD   # move main to the top
# real Git shortcut:
git commit --fixup=<sha> && git rebase -i --autosquash <base>
```

```
C0-C1 (main)-C2 (newImage)-C3 (caption*)
      goal: change C2, keep order, main on top
```

Where it is mostly used

Fixing a mistake in an earlier commit of your feature branch before review, e.g. a reviewer asks for a change in commit 2 of 5.

Common mistakes

- Amending when HEAD is on the wrong commit - amend always changes the **current** commit.
- Forgetting amend creates a **new** commit with a new ID.
- Reordering causes conflicts if commits touch the same lines.
- Leaving main behind at the old position.

Level hint: Primes (C2") in the goal show how many times a commit was copied. Order matters; exact primes are not strictly checked.

4.3 Juggling Commits #2

Site level: *Juggling Commits #2*

What it is

The same problem solved with **cherry-pick** instead of rebase: build the desired history on main directly.

Commands and example

```
git checkout main
git cherry-pick C2
git commit --amend
git cherry-pick C3
```

C0-C1-C2''-C3' (main*) old C2, C3 remain on their branches

Where it is mostly used

Same as 4.2 - choose cherry-pick when it's clearer and involves fewer rewrites.

Common mistakes

- Cherry-picking in the wrong order.
- Amending before picking the right commit.
- Not realising the originals remain on their branch - clean up if needed.

Level hint: Think: 'pick, amend, pick'.

4.4 Git Tags

Site level: *Git Tags*

What it is

A **tag** is a permanent label on a commit. Unlike branches, tags **don't move** when you commit. Lightweight tags are just names; annotated tags (-a) also store author, date and message.

Commands and example

```
git tag v1 C1                    # lightweight
git tag -a v1.0 -m "Release 1.0" # annotated (recommended)
git tag                        # list
git checkout v1                # detached HEAD at v1
git push origin v1.0           # tags aren't pushed by default
git push --tags
```

C0-C1(v1)-C2-C3 (main*) v1 stays at C1 forever

Where it is mostly used

Marking releases (v2.3.0), milestones, deployments. CI/CD systems often build and publish when a version tag is pushed; GitHub Releases are based on tags.

Common mistakes

- Forgetting to push tags - teammates and CI never see them.
- Moving or deleting a published tag; others still have the old one.
- Committing after checking out a tag (you're detached - create a branch first).
- Using lightweight tags for releases when annotated tags carry more information.
- Inconsistent naming (v1, 1.0, release-1). Follow semantic versioning.

Level hint: Tags can be placed on any commit by ID; you may also need to check one out.

4.5 Git Describe

Site level: *Git Describe*

What it is

`git describe` tells you where a commit is relative to the **nearest tag**. Output format:

`<tag>-<numCommits>-g<hash>` (the site shows underscores). If the commit is tagged, it prints just the tag.

Commands and example

```
git describe main      # v1-2-gC2  -> 2 commits after v1, at C2
git describe v1        # v1
git describe --tags    # real Git: include lightweight tags
```

```
C0-C1(v1)-C2-C3 (main)
  git describe main -> v1-2-gC3
```

Where it is mostly used

Generating version strings for builds (e.g. app shows 'v2.1-14-g3f9a2b1'), and quickly knowing how far a commit is from the last release.

Common mistakes

- In real Git, by default only **annotated** tags count - lightweight tags need `--tags`, or describe fails with 'No names found'.
- Misreading the number as 'commits until the tag' - it's commits **since** the tag.
- The 'g' is a prefix meaning 'git', not part of the hash.

Level hint: The level completes after you run describe a few times and commit - just explore.

Main 5 - Advanced Topics

Puzzles that stretch your understanding: chained rebases, navigating merge parents, and rearranging several branches.

5.1 Rebasing over 9000 times

Site level: *Rebasing over 9000 times*

What it is

Several feature branches need to end up **in one line, in a specific order**, with main at the tip. Use the two-argument rebase repeatedly: each branch is rebased onto the previous one, then main is fast-forwarded.

Commands and example

```
git rebase main bugFix
git rebase bugFix side
git rebase side another
git rebase another main # fast-forward main to the top
```

Result: C0-C1-C3'-C4'-C5'-C6'-C7' (main, another, side, bugFix at their points)

Where it is mostly used

Stacking related feature branches ('stacked PRs'), preparing a series of changes for a single release with linear history.

Common mistakes

- Rebasing in the wrong order, producing a different sequence than required.
- Forgetting the two-argument form and rebasing from the wrong checked-out branch.
- Leaving main behind at the end.
- Ending with duplicated work because a branch was rebased onto a base that already contained its commits (Git usually skips identical patches, but be careful).

Level hint: Plan the final order first, then rebase each branch onto the one that should come before it.

5.2 Multiple parents

Site level: *Multiple parents*

What it is

A merge commit has **two parents**. $\wedge$ alone follows the **first** parent (the branch you were on when merging). $\wedge 2$ follows the **second** parent (the branch that was merged in). You can chain modifiers: HEAD~ $\wedge 2$ ~2.

Commands and example

```
git checkout HEAD^      # first parent
git checkout HEAD^2     # second parent
git checkout HEAD~ $\wedge 2$ ~2 # back one, second parent, back two
git branch bugWork HEAD~ $\wedge 2$ ~
```

```

      C2 ---- C4 (merge)
     /      /
C0-C1      /      C4^ = C2
     \      /      C4^2 = C3
      C3 -
```

Where it is mostly used

Investigating history of merged features, `git log/git diff` across merges, and reverting merges (`git revert -m 1`).

Common mistakes

- Thinking $^2 = \sim 2$.
- Assuming parent order is random - first parent is always the branch that received the merge.
- Writing long chains without checking each step; test them incrementally.

Level hint: You can create a branch at a relative ref in one command without checking it out.

5.3 Branch Spaghetti

Site level: *Branch Spaghetti*

What it is

Three branches (one, two, three) must each contain particular commits from main in a particular order. It combines **cherry-pick** with **branch forcing**.

Commands and example

```
git checkout one
git cherry-pick C4 C3 C2    # order matters
git checkout two
git cherry-pick C5 C4 C3 C2
git branch -f three C2      # just point it
```

Each branch ends with its own sequence of copied commits.

Where it is mostly used

Preparing different release lines or customer-specific branches from shared work.

Common mistakes

- Over-complicating: sometimes a branch only needs to be pointed (`branch -f`), not rebuilt.
- Wrong order of cherry-picks.
- Forgetting which branch HEAD is on before cherry-picking.

Level hint: Read the goal graph carefully; compare order of commits per branch.

Remote 1 - Push & Pull: Git Remotes

Remotes are copies of your repository on another computer (GitHub, GitLab, a server). These levels teach how local and remote repositories stay in sync. On the site, the remote is drawn with dashed lines and remote branches are shown as o/main (in real Git: origin/main).

R1.1 Clone Intro

Site level: *Clone Intro*

What it is

`git clone` copies a remote repository to your machine, sets up a remote named **origin**, and creates local branches plus remote-tracking branches.

Commands and example

```
git clone https://github.com/user/repo.git
git clone git@github.com:user/repo.git    # SSH
git remote -v                             # show remotes
```

```
Remote: C0-C1 (main)      Local: C0-C1 (main*, o/main)
```

Where it is mostly used

Starting work on any existing project, open-source contributions, setting up a new machine, CI servers.

Common mistakes

- Cloning inside another repository folder (nested repos).
- HTTPS vs SSH confusion and authentication failures (GitHub needs a token or SSH key, not your password).
- Cloning someone else's repo when you meant to fork it first (to contribute).

Level hint: Just run `git clone` - the site creates the simulated remote.

R1.2 Remote Branches

Site level: *Remote Branches*

What it is

Remote-tracking branches like `o/main` record **where the remote's branch was the last time you communicated**. You cannot move them by committing; checking one out puts you in detached HEAD. They only update when you fetch/pull/push.

Commands and example

```
git checkout o/main    # detached HEAD
git commit             # new commit, o/main does NOT move
git branch -r          # list remote-tracking branches
```

```
C0-C1 (main, o/main)-C2 (HEAD)    o/main stays at C1
```

Where it is mostly used

Seeing how your local work compares to the server: `git log main..origin/main`, `git status` 'ahead/behind' messages.

Common mistakes

- Committing on o/main expecting to update the remote.
- Believing origin/main is always up to date - it's a snapshot from your last fetch.
- Confusing origin/main (local record) with origin main (remote + branch arguments).

Level hint: Watch o/main stay put while HEAD moves.

R1.3 Git Fetchin'

Site level: *Git Fetchin'*

What it is

`git fetch` downloads commits the remote has that you don't, and updates your remote-tracking branches (o/main). It does **not** change your local branches or files.

Commands and example

```
git fetch
git fetch origin
git log --oneline main..origin/main # what's new?
```

```
Before: local C0-C1 (main, o/main)    remote C0-C1-C2-C3 (main)
After : local C0-C1 (main)-C2-C3 (o/main)
```

Where it is mostly used

Safely checking what teammates pushed before deciding to merge/rebase; code review; CI.

Common mistakes

- Thinking fetch updates your main or working files - it doesn't.
- Forgetting to fetch before comparing branches, giving stale results.

Level hint: Only o/ branches move.

R1.4 Git Pullin'

Site level: *Git Pullin'*

What it is

`git pull` = `git fetch` + `git merge` of the remote-tracking branch into your current branch. `git pull --rebase` = `fetch` + `rebase`.

Commands and example

```
git pull # fetch + merge o/main
git pull --rebase # fetch + rebase
# equivalent manual steps:
git fetch; git merge o/main
```

```
local C0-C1-C3 (main*)  remote has C2
after pull: merge commit C4 with parents C3 and C2
```

Where it is mostly used

Starting your day, before pushing, keeping a branch current with the team.

Common mistakes

- Pulling with uncommitted changes (commit or `git stash` first).
- Creating many noisy merge commits - consider `pull --rebase` or `git config pull.rebase true`.
- Pulling while on the wrong branch.

Level hint: Pull is just a shortcut you can reproduce manually.

R1.5 Faking Teamwork

Site level: *Faking Teamwork*

What it is

Site-only command. `git fakeTeamwork` simulates a teammate pushing commits to the remote, so you can practise syncing.

Commands and example

```
git fakeTeamwork          # 1 commit on remote main
git fakeTeamwork foo 3    # 3 commits on remote foo
```

Remote gets new commits; your local repo doesn't know yet.

Where it is mostly used

Only on the site. In real life, teammates push actual commits.

Common mistakes

- Trying `git fakeTeamwork` in a real terminal - it doesn't exist.

Level hint: Simulate, then fetch or pull to see the effect.

R1.6 Git Pushin'

Site level: *Git Pushin'*

What it is

`git push` uploads your commits to the remote and updates the remote branch plus your o/main. Without arguments it pushes according to your branch's upstream/push settings.

Commands and example

```
git push
git push origin main
git push -u origin feature  # set upstream on first push
```

local C0-C1-C2 (main*) o/main at C1 -> after push: o/main at C2, remote main at C2

Where it is mostly used

Sharing work, backing up to GitHub, opening pull requests, triggering CI/CD.

Common mistakes

- Pushing without pulling first -> rejected.
- Pushing to the wrong branch.
- Using `git push --force` and overwriting teammates' commits. Prefer `--force-with-lease`.
- Pushing secrets - once pushed, treat them as compromised.

Level hint: After pushing, local and remote graphs match.

R1.7 Diverged History

Site level: *Diverged History*

What it is

If the remote has commits you don't, and you have commits it doesn't, Git **rejects your push**. You must first incorporate the remote work - by rebase or merge - then push.

Commands and example

```
# rebase approach
git fetch
git rebase o/main
git push
# shortcut
git pull --rebase; git push
# merge approach
git pull; git push
```

```
remote: C0-C1-C2      local: C0-C1-C3
after pull --rebase: C0-C1-C2-C3'  -> push OK
```

Where it is mostly used

Every team daily: two people working on the same branch.

Common mistakes

- Using force push to 'fix' the rejection - destroys others' work.
- Not resolving conflicts properly during rebase/merge.
- Misreading the error 'Updates were rejected because the remote contains work that you do not have locally'.

Level hint: The level accepts either strategy.

R1.8 Locked Main

Site level: *Locked Main*

What it is

Many teams **protect main**: direct pushes are rejected; changes must come via pull requests. If you committed on main locally by mistake, move the work to a feature branch, push that, and reset local main to match the remote.

Commands and example

```
git checkout -b feature      # branch at your commit
git push origin feature
git branch -f main o/main    # or: checkout main; git reset --hard o/main
# then open a Pull Request
```

```
local main had C2 -> now feature points at C2, main back at o/main
```

Where it is mostly used

Almost every professional repository (branch protection rules, required reviews and CI checks).

Common mistakes

- Committing directly on main locally out of habit.
- Resetting main before saving the commit on another branch (losing work).
- Asking an admin to disable protection instead of using a PR.

Level hint: Save the work on a branch first, then fix main.

Remote 2 - To Origin and Beyond: Advanced Remotes

Fine control over what goes where: integrating multiple branches, tracking relationships, and the full argument syntax of push, fetch and pull.

R2.1 Push Main!

Site level: *Push Main!*

What it is

Several local feature branches must be integrated into the remote main, which has moved on. Rebase each feature onto the latest o/main in order, then push.

Commands and example

```
git fetch
git rebase o/main side1
git rebase side1 side2
git rebase side2 side3
git rebase side3 main
git push
```

Result: linear history on remote main containing all features

Where it is mostly used

Integrating stacked work, maintainers landing several contributions.

Common mistakes

- Forgetting to fetch first.
- Rebasing in the wrong order.
- Pushing before main points at the final commit.

Level hint: Same idea as 5.1 plus the remote.

R2.2 Merging with remotes

Site level: *Merging with remotes*

What it is

The same integration done with **merge** instead. The level also discusses the debate: rebase gives clean linear history; merge preserves true history (who did what, when).

Commands and example

```
git checkout main
git pull
git merge side1
git merge side2
git merge side3
git push
```

Result: remote main with merge commits for each feature

Where it is mostly used

Teams that value exact history, audit trails, or use GitHub's default merge button.

Common mistakes

- Mixing strategies inconsistently within a team.
- Forgetting to pull before merging.

Level hint: Compare your final graph with the rebase version.

R2.3 Remote Tracking

Site level: *Remote Tracking*

What it is

Local branches can **track** a remote branch (the 'upstream'). That's why plain `git pull`/`git push` know where to go. Any branch can track any remote branch.

Commands and example

```
git checkout -b foo o/main      # foo tracks o/main
git branch -u o/main foo       # set tracking for foo
git branch -u o/main           # for current branch
git branch -vv                 # show tracking info
git push -u origin foo         # push and set upstream
```

```
foo -> tracks o/main; git push from foo updates remote main
```

Where it is mostly used

Working on branches with different local names, setting up a new branch's upstream on first push.

Common mistakes

- Error 'The current branch has no upstream branch' - use `git push -u`.
- Accidentally tracking the wrong remote branch, so pushes go somewhere unexpected.

Level hint: Tracking decides where a branch's commits go on push.

R2.4 Git push arguments

Site level: *Git push arguments*

What it is

`git push <remote> <place>`: take commits from local branch *place* and update the branch of the same name on the remote. It works **regardless of where HEAD is**.

Commands and example

```
git push origin main
git push origin foo
```

```
HEAD may be anywhere; the named branch is what gets pushed
```

Where it is mostly used

Pushing a specific branch without checking it out; scripts.

Common mistakes

- Assuming it pushes the current branch - it pushes the one you name.
- Typos in branch names create new remote branches.

Level hint: Check that HEAD didn't need to move.

R2.5 Git push arguments - Expanded

Site level: *Git push arguments -- Expanded!*

What it is

Full refspec form: `git push origin <source>:<destination>`. Source can be any commit reference; destination is the remote branch name. If the destination doesn't exist, Git creates it.

Commands and example

```
git push origin foo^:main      # push parent of foo to remote main
git push origin main:newBranch # creates remote newBranch
git push origin HEAD:refs/for/main # Gerrit-style review
```

Where it is mostly used

Pushing a partial branch, publishing under a different name, code review systems like Gerrit.

Common mistakes

- Overwriting a remote branch you didn't intend.
- Swapping source and destination.
- Pushing a commit that isn't a descendant of the remote branch -> rejected.

Level hint: Source is local, destination is remote.

R2.6 Fetch arguments

Site level: *Fetch arguments*

What it is

`git fetch origin foo` fetches only foo and updates o/foo. With a refspec `git fetch origin <source>:<destination>` the source is on the remote and destination is a **local** branch, updated directly. You can't fetch into the currently checked-out branch.

Commands and example

```
git fetch origin foo      # updates o/foo only
git fetch origin foo~1:bar # local bar = remote foo~1
git fetch                 # everything
```

Where it is mostly used

Fetching one large branch only, updating a local branch without checking it out.

Common mistakes

- Expecting `git fetch origin foo` to update local foo.
- Fetching into the currently checked-out branch - Git refuses.
- Reversed direction compared to push refsspecs is confusing: remote first for fetch.

Level hint: In fetch, source is on the remote.

R2.7 Source of nothing

Site level: *Source of nothing*

What it is

An empty source has special meaning. `git push origin :foo` **deletes** remote branch foo. `git fetch origin :bar` **creates** a new local branch bar.

Commands and example

```
git push origin :foo           # delete remote foo
git push origin --delete foo    # clearer modern form
git fetch origin :bar          # create local bar
```

Where it is mostly used

Cleaning up merged remote branches after a PR.

Common mistakes

- Accidentally deleting a remote branch via a stray colon.
- Prefer the explicit `--delete` for readability.

Level hint: Weird but short level.

R2.8 Pull arguments

Site level: *Pull arguments*

What it is

`git pull origin foo=git fetch origin foo; git merge o/foo.git pull origin bar:bugFix=git fetch origin bar:bugFix; git merge bugFix`. The merge always goes into your **current** branch.

Commands and example

```
git pull origin foo
git pull origin bar:bugFix
```

Where it is mostly used

Pulling a teammate's specific branch into yours for testing.

Common mistakes

- Being on the wrong branch - the merge lands where HEAD is.
- Expecting the destination branch to be the one merged into (it's merged *from*).

Level hint: Translate pull into fetch + merge in your head.

Appendix A - Merge vs Rebase

	Merge	Rebase
History shape	Branching graph with merge commits	Straight line
Rewrites commits?	No - safe on shared branches	Yes - new commit IDs
Shows true timeline?	Yes	No - looks sequential
Conflicts	Resolved once, in the merge commit	May need resolving per replayed commit
Best for	Integrating finished features into main; shared branches	Updating your private feature branch; tidy PRs
Undo	git revert -m 1 <merge>	git reflog + git reset to the old commit

Rule of thumb: rebase your own local work to keep it tidy; merge (or use the PR button) to bring work into shared branches. Never rebase what others already pulled.

Appendix B - Recovery toolkit

Mistakes happen. These commands get you out of almost every situation covered in this guide.

```
git status                # always start here - where am I, what changed?
git log --oneline --graph --all # see the whole graph (like the site!)
git reflog                # every place HEAD has been - find 'lost' commits
git branch rescue <sha>   # put a branch on a lost commit
git merge --abort         # cancel a conflicted merge
git rebase --abort        # cancel a rebase in progress
git cherry-pick --abort   # cancel a cherry-pick
git reset --hard ORIG_HEAD # undo the last merge/rebase (local only)
git stash / git stash pop  # park uncommitted changes temporarily
git restore <file>         # discard edits to a file
git restore --staged <file> # unstage a file
git push --force-with-lease # force push only if nobody else pushed meanwhile
```

Remember: committed work is very hard to lose permanently (reflog keeps it ~90 days by default).

Uncommitted work destroyed by reset --hard or restore is gone. Commit early.

Appendix C - Command cheat sheet

Task	Command
Create commit	<code>git commit -m "msg"</code>
Create branch	<code>git branch name</code>
Switch branch	<code>git checkout name / git switch name</code>
Create + switch	<code>git checkout -b name / git switch -c name</code>
Merge X into current	<code>git merge X</code>
Rebase current onto X	<code>git rebase X</code>
Rebase B onto A	<code>git rebase A B</code>
Detach HEAD	<code>git checkout <commit></code>
Parent / N back	<code>HEAD^ HEAD~N</code>
Second parent of merge	<code>HEAD^2</code>
Move a branch	<code>git branch -f name <commit></code>
Undo local commit	<code>git reset HEAD~1</code>
Undo shared commit	<code>git revert <commit></code>
Copy commits here	<code>git cherry-pick C1 C2</code>
Edit recent commits	<code>git rebase -i HEAD~N</code>
Change last commit	<code>git commit --amend</code>
Tag a commit	<code>git tag v1 <commit></code>
Describe position	<code>git describe <ref></code>
Clone	<code>git clone <url></code>
Download only	<code>git fetch</code>
Download + integrate	<code>git pull / git pull --rebase</code>
Upload	<code>git push</code>
Set upstream	<code>git branch -u origin/main foo / git push -u origin foo</code>
Push with refspec	<code>git push origin <src>:<dst></code>
Fetch with refspec	<code>git fetch origin <src>:<dst></code>
Delete remote branch	<code>git push origin :foo / git push origin --delete foo</code>
Pull specific branch	<code>git pull origin foo</code>

Appendix D - Self-test questions

Try answering before reading the answer. If you struggle, revisit the matching section and replay the level in sandbox mode.

Q1. You run git branch feature and then git commit. Where does the commit go?

Answer: To your current branch - git branch does not switch. Use git checkout -b / git switch -c.

Q2. What is the difference between git merge and git rebase?

Answer: Merge joins histories with a merge commit (no rewriting). Rebase copies your commits onto a new base for linear history (rewrites).

Q3. Why shouldn't you rebase a branch others have pulled?

Answer: Rebase creates new commit IDs; others' copies diverge and need painful fixes.

Q4. What does HEAD~3 mean? And HEAD^2?

Answer: Three first-parent steps back. HEAD^2 is the second parent of a merge commit.

Q5. You pushed a bad commit to main. reset or revert?

Answer: revert - it adds an undo commit without rewriting shared history.

Q6. What does git fetch change on your machine?

Answer: Only remote-tracking branches (origin/*) and downloaded objects - not local branches or files.

Q7. git pull is equivalent to which two commands?

Answer: git fetch followed by git merge (or git rebase with --rebase).

Q8. Your push is rejected: 'remote contains work you do not have'. What now?

Answer: git pull --rebase (or git pull), resolve conflicts, then git push. Don't force push.

Q9. How do you change a commit that is 3 commits back?

Answer: git rebase -i (edit/reorder + amend) or git commit --fixup + rebase --autosquash, or cherry-pick + amend.

Q10. Why doesn't anyone see your new tag?

Answer: Tags aren't pushed by default: git push origin <tag> or git push --tags.

Q11. What does git push origin foo^:main do?

Answer: Updates remote main to the parent of your local foo.

Q12. What does git push origin :bugfix do?

Answer: Deletes the remote branch bugfix.

Q13. You committed in detached HEAD and switched away. How do you get the commit back?

Answer: git reflog to find it, then git branch rescue <sha>.

Q14. Main is protected and you committed on it locally. Fix?

Answer: Create a branch at the commit, push that branch, reset local main to origin/main, open a PR.

Suggested practice plan: Day 1 - Main 1 and 2. Day 2 - Main 3 and 4. Day 3 - Main 5. Day 4 - Remote 1. Day 5 - Remote 2. Day 6 - redo any level you needed hints for, then recreate the scenarios in a real repository using git log --oneline --graph --all.